

VISITOR APPROVAL ENTRY SYSTEM

A full-stack visitor registration and approval platform — React, Node/Express & MySQL

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Y25 · 2026–2027 · Trimester 4 | Koneru Lakshmaiah Education Foundation

D. Shamika Hasini (2520080033)

M. Priyavallika (2520090183)

Abstract

- 1 What it is**

A digital gate-security app for apartment/office visitor entry, with separate Guard and Tenant portals.
- 2 Problem solved**

Replaces manual visitor registers and unverified walk-ins with a logged, approval-based entry process.
- 3 Core flow**

Guard registers a visitor → request sent to the tenant → tenant approves/denies → guard sees the result.
- 4 Tech used**

React (Vite) frontend, Node.js/Express REST API, MySQL database with bcrypt-hashed passwords.
- 5 Benefit**

Faster, traceable visitor handling with a permanent visit log and role-based access.

Problem Statement

Manual registers

Paper logs are slow, illegible, and easy to falsify or lose.

No approval step

Walk-in visitors often enter before the resident/host confirms them.

Poor traceability

No searchable history of who visited, when, or why.

No absence handling

Manual systems can't verify a visitor when the host is away.

A digital, role-based approval workflow is needed to close these gaps.

Existing Approach	Method	Limitation	VAES Improvement
Paper visitor register	Handwritten log at gate	Slow, illegible, no approval step	Digital form + instant tenant approval/denial
Gate phone call to resident	Guard calls resident to confirm	No record kept; depends on call being answered	Absence-password fallback + stored visit log
Temporary approval generator	No storage	History is not recorded	Web-based, role-separated Guard/Tenant dashboards with history

Research gap: most classroom/reference visitor-management approaches log entries but lack a real-time, host-side approve/deny workflow with role separation the gap VAES targets.

Note: comparisons reflect common manual/legacy approaches; no specific published papers were provided in the project files.

Objectives

- 1 Register visitor details digitally (name, phone, purpose, host block/flat)
- 2 Route each visit request to the tenant for approval or rejection
- 3 Support pre-approval of expected guests, so entry is instant
- 4 Allow tenants to schedule expected visits in advance
- 5 Maintain a permanent, searchable visit log for every entry
- 6 Provide secure, role-based login for guards and tenants

Proposed System

VAES is a two-portal web app (Guard / Tenant) backed by a REST API and MySQL database.

1

Authentication

Login & signup for tenants/guards; bcrypt password hashing

2

Security Module

Guard dashboard: register visitors, view sent requests

3

Tenant Module

Tenant dashboard: approvals, pre-approvals, schedules

4

Visitor Management

Visitor entry form; via Pre-Approved / Schedule / Direct

5

Approval Management

Tenant approves/denies pending visitor requests

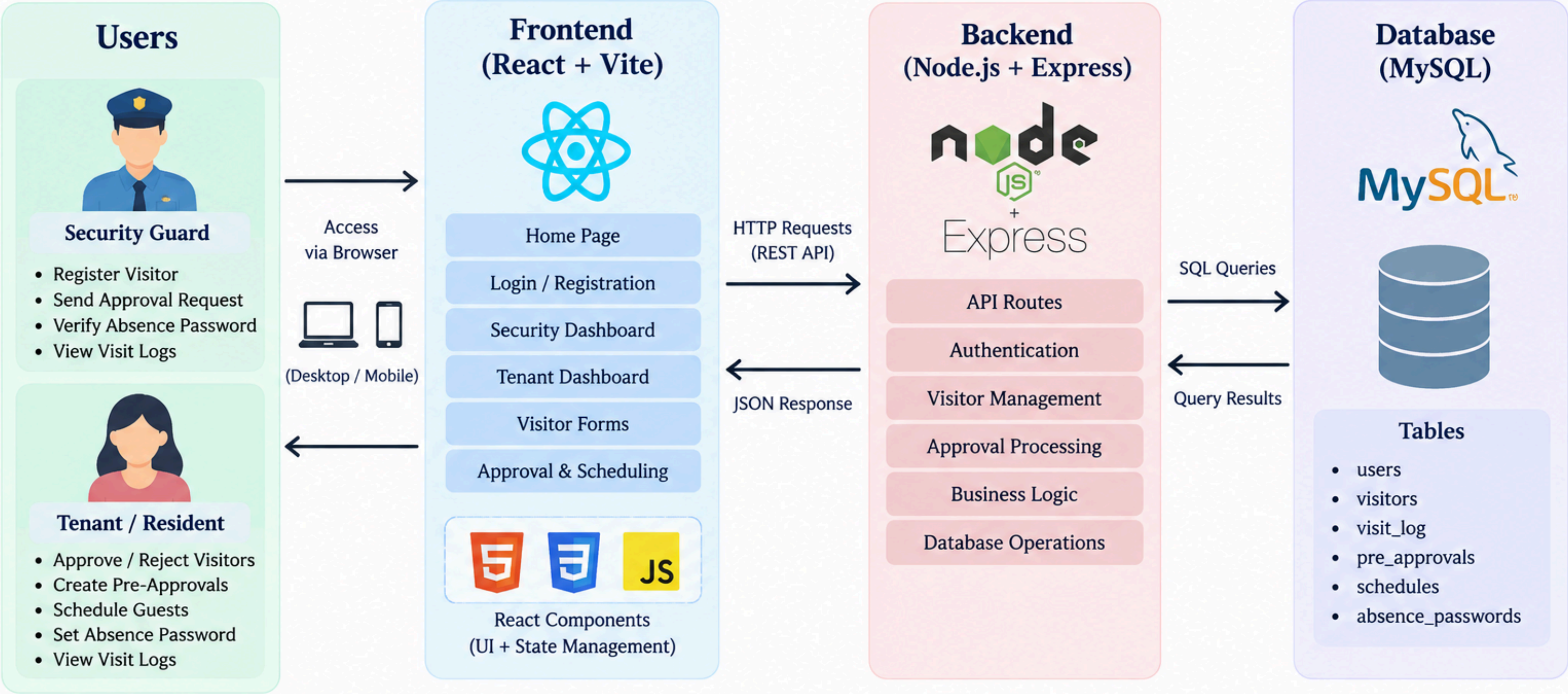
6

Visit Logs

Append-only historical log of every visitor record

System Architecture

Visitor Approval Entry System (VAES)



Technology Stack



React
(Vite)



Node.js
(Express)



MySQL
(Database)



HTML5



CSS3



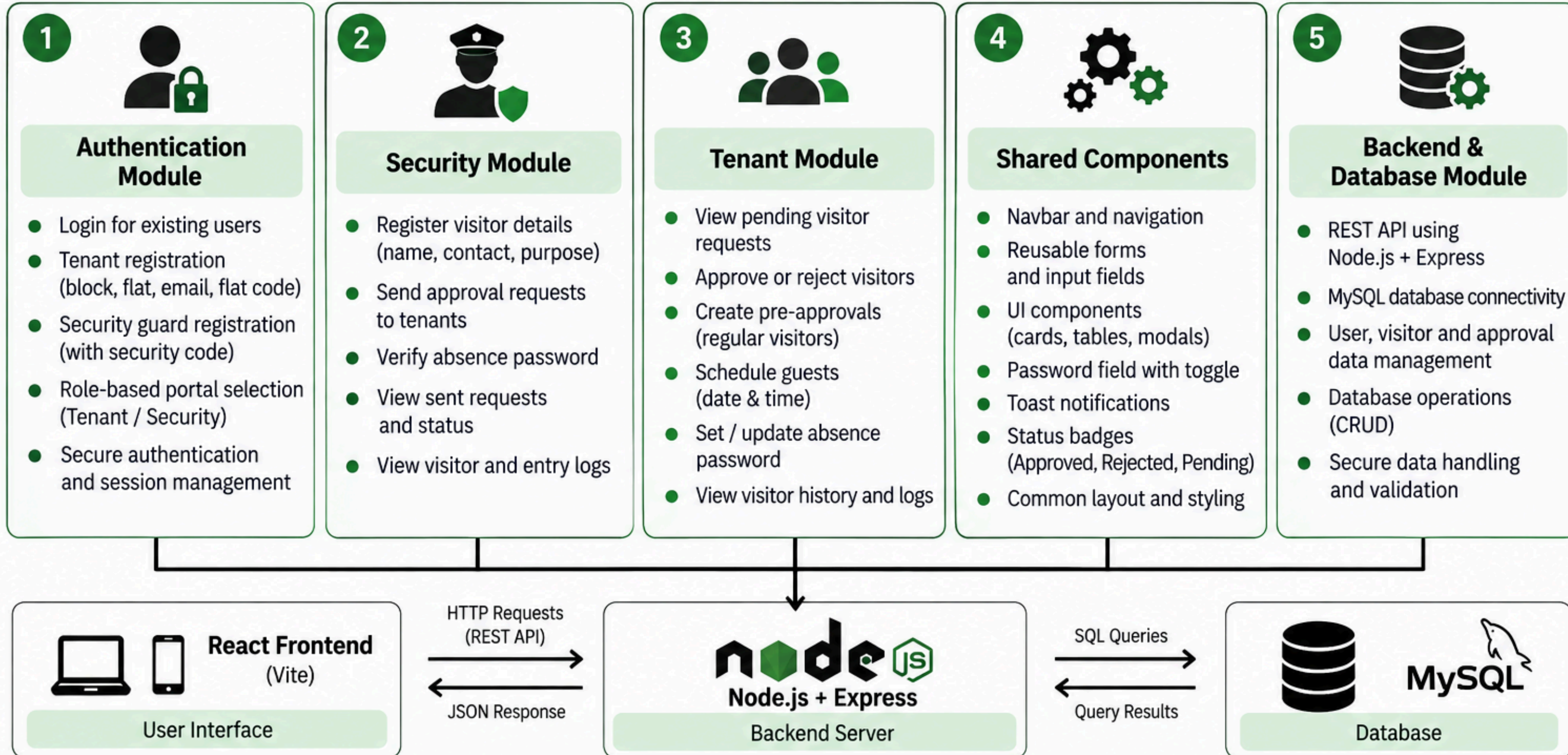
JavaScript



Visual Studio Code
(Development)

A Secure and Efficient Digital Solution for Visitor Management

Key Modules of VAES



Integrated Modules



Secure Access



Efficient Visitor Management



Better Communities

FRONTEND DESIGN

Home Page

- Introduces the Visitor Approval Entry System (VAES)
- Provides navigation to the system
- Clean and user-friendly interface

Login / Registration

- Tenant registration
- Security registration
- Secure authentication

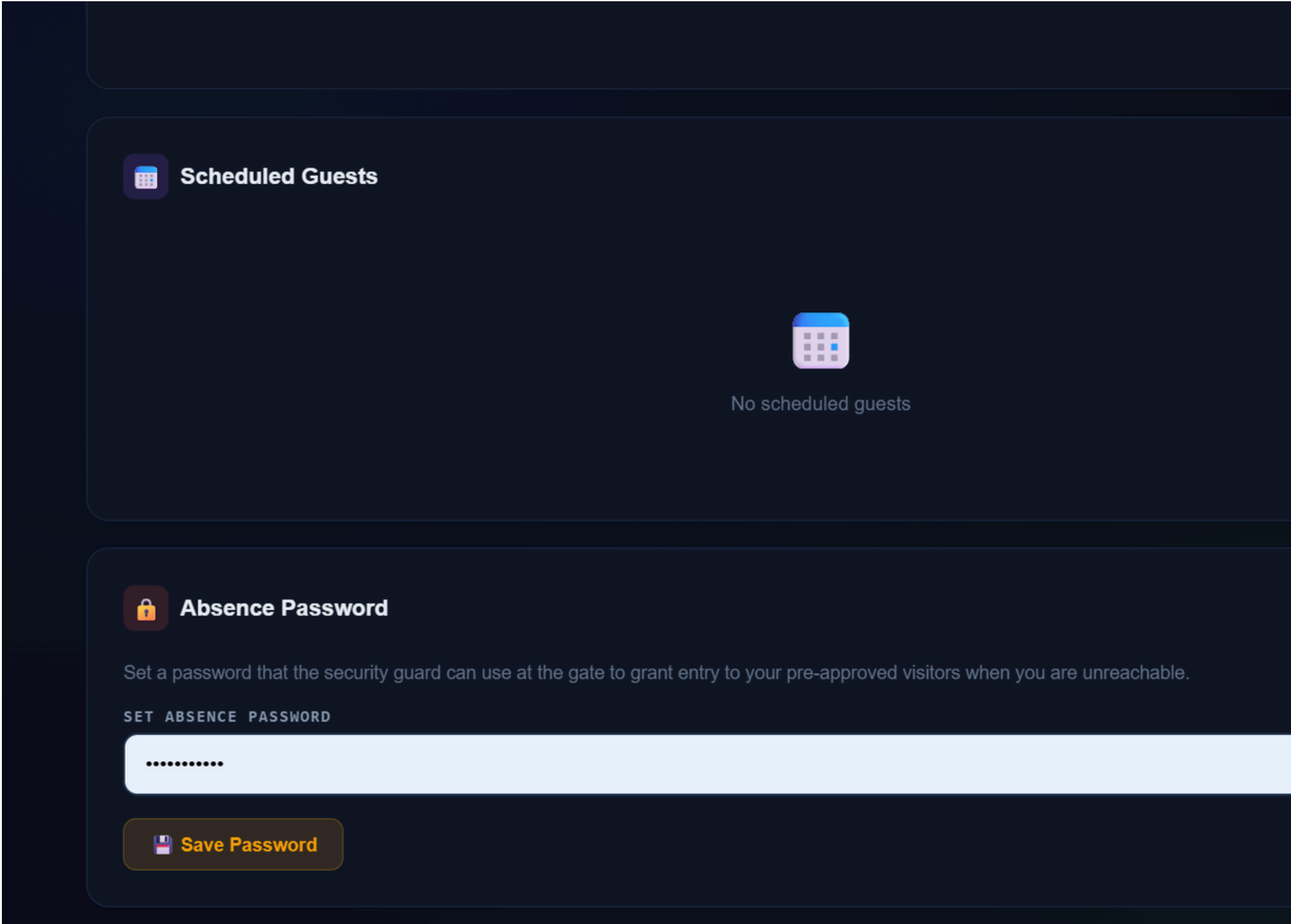
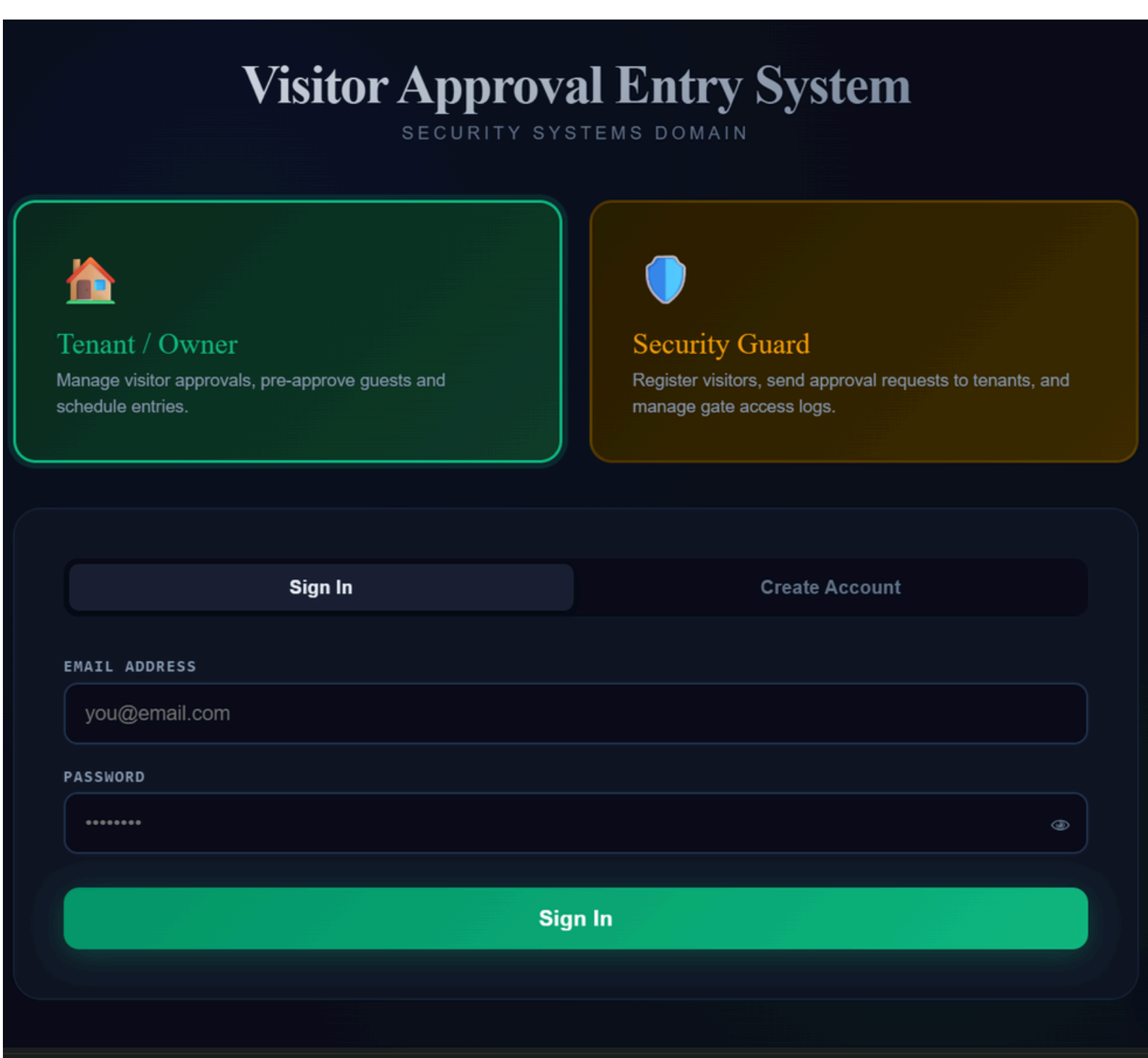
forms

- **Visitor registration form**
- **Visitor details: name, contact, purpose, host**
- **Pre-approval and guest scheduling forms**
- **Input validation**

Dashboard

- Security Dashboard: visitor registration, approval requests and visit logs
- Tenant Dashboard: approve/reject requests, pre-approvals, scheduling and visit history

FRONTEND Implementation Screenshots



Database Design

MySQL database "vaes" — 6 tables, linked mainly through email / schedule_id

users

- PK id
- role (tenant/guard)
- email (UNIQUE)
- password (bcrypt)
- block, flat

absence_passwords

- PK/FK user_email
- password (bcrypt)
- → users.email

pre_approvals

- PK id
- name, phone, relation
- block, flat

schedules

- PK id
- name, phone, date/time
- block, flat

visitors

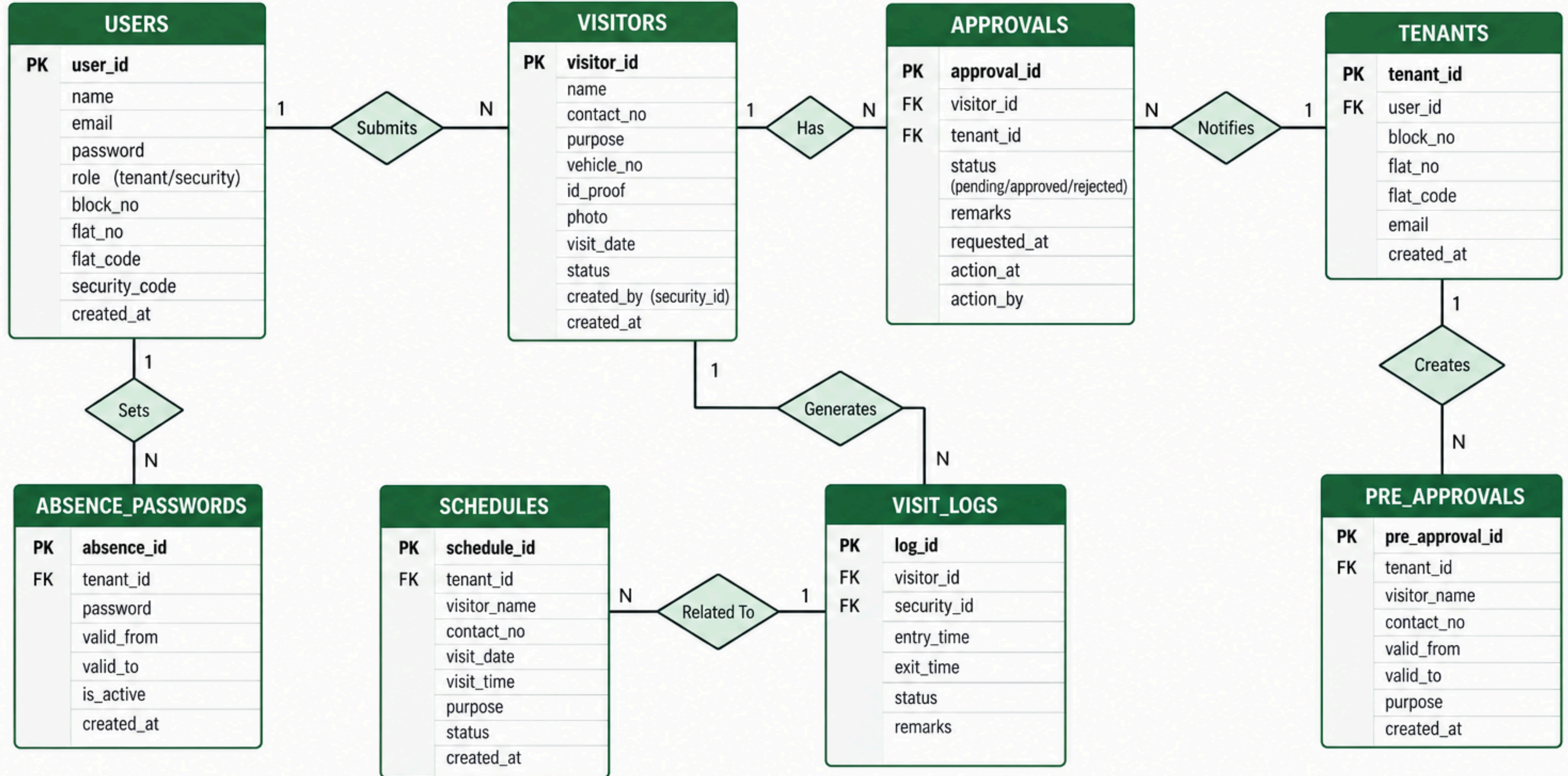
- PK id
- status
(pending/approved/denied)
- FK schedule_id
- block, flat, guard_id

visit_log

- PK id (append-only)
- same fields as visitors
- permanent history

Entity Relationship Diagram

Visitor Approval Entry System (VAES)



Database Tables/Schema

Result Grid			Filter Rows:		Edit:				Export/Import:			Wrap Cell Content:	
	id	name	phone	relation	until	schedule	block	flat	created_at				
▶	9000000000001	Ravi Kumar	9876543210	Brother	2026-12-31		A	101	2026-09-11 14:48:40				
	9000000000002	Anjali Rao	9123456780	Friend	2026-10-15		A	101	2026-09-11 14:48:40				
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL				

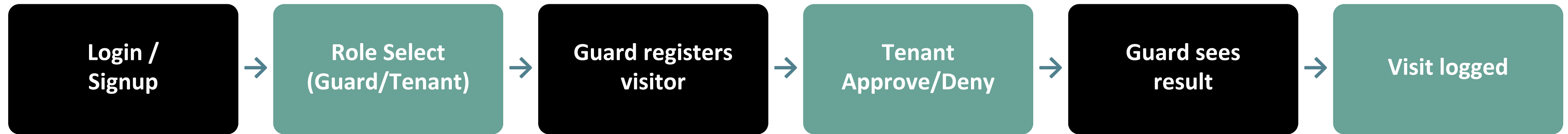
Result Grid			Filter Rows:		Edit:				Export/Import:			Wrap Cell Content:	
	id	name	phone	date	time	purpose	block	flat	created_at				
▶	9000000000101	Amazon Delivery	9000011111	2026-09-15	14:00	Package delivery	A	101	2026-09-11 14:48:40				
	9000000000102	Plumber - Suresh	9000022222	2026-09-18	10:30	Leak repair	B	402	2026-09-11 14:48:40				
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL				

Result Grid			Filter Rows:		Edit:				Export/Import:			Wrap Cell Content:	
	id	name	phone	purpose	block	flat	status	via	time	guard_id	schedule_id		
▶	9000000000201	Deepak Sharma	9988776655	Meeting friend	A	101	approved	Host Approved	11/9/2026, 2:48:40 pm	ramya@gmail.com	NULL		
	9000000000202	Unknown Visitor	9111222333	Sales pitch	C	202	denied	Host Denied	11/9/2026, 2:48:40 pm	ramya@gmail.com	NULL		
	9000000000203	Courier - Swiggy Genie	9333444555	Package drop	B	402	pending	Awaiting Approval	11/9/2026, 2:48:40 pm	ramya@gmail.com	NULL		
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL		

Challenges / Issues Faced

- **React conversion –**
Converting the existing frontend into reusable React components.
- **Frontend–Backend Integration –**
Connecting React with the Node.js/Express REST API.
- **MySQL Connectivity –**
Configuring database connection and environment variables.
- **Database Setup –**
Creating tables, relationships and inserting sample records.
- **State Management –**
Managing shared user, visitor and approval data using React Context.
- **Data Validation –**
Handling form inputs and preventing invalid visitor information.
- **Debugging –**
Resolving errors during frontend, backend and database integration.

System Workflow



Alternate / supporting flows in code:

Pre-Approval

Tenant adds guest in advance (AddPreApprovalModal) → guard entry marked "Pre-Approved" → instant access, no approval wait.

Scheduling

Tenant creates a scheduled visit (AddScheduleModal, date/time/purpose) → linked to a visitor request via `schedule_id`.

Absence Password

Tenant sets an absence password (AbsencePasswordCard) → guard verifies visitor via `VerifyAbsencePasswordCard` when tenant is unreachable.

Project Structure

```
vaes-react/  
├─ src/  
│   ├─ App.jsx           - routes Guard vs Tenant view  
│   ├─ main.jsx          - React root entry point  
│   ├─ api.js            - fetch wrappers for backend REST API  
│   ├─ context/AppContext.jsx - global state (auth, visitors, toasts)  
│   ├─ components/  
│   │   ├─ auth/         - LoginForm, SignupTenantForm, SignupGuardForm  
│   │   ├─ security/     - SecurityDashboard, RegisterVisitorCard, GuardVisitLogPanel  
│   │   ├─ tenant/       - TenantDashboard, ApprovalRequestsPanel, PreApprovalsCard, etc.  
│   │   └─ shared/       - Modal, Navbar, StatCard, StatusBadge, Toast  
│   └─ data/hardcodedUsers.js  
├─ server/  
│   ├─ index.js          - Express app & all API routes  
│   ├─ db.js             - MySQL connection pool (mysql2)  
│   ├─ schema.sql        - table definitions  
│   └─ seed.js           - sample data loader  
└─ package.json
```

Future Scope

Not implemented in the current project — proposed enhancements only:

- 1 Facial Recognition – Verify visitors using facial identification
- 2 QR Code Verification – Provide quick and secure visitor entry.
- 3 Mobile application (Android/iOS)
- 4 Cloud deployment & scaling

Results / Benefits

1 Faster processing

Digital form + instant pre-approval removes manual write-ups at the gate

2 Centralized records

All visitor, approval, and log data stored in one MySQL database

3 Better security

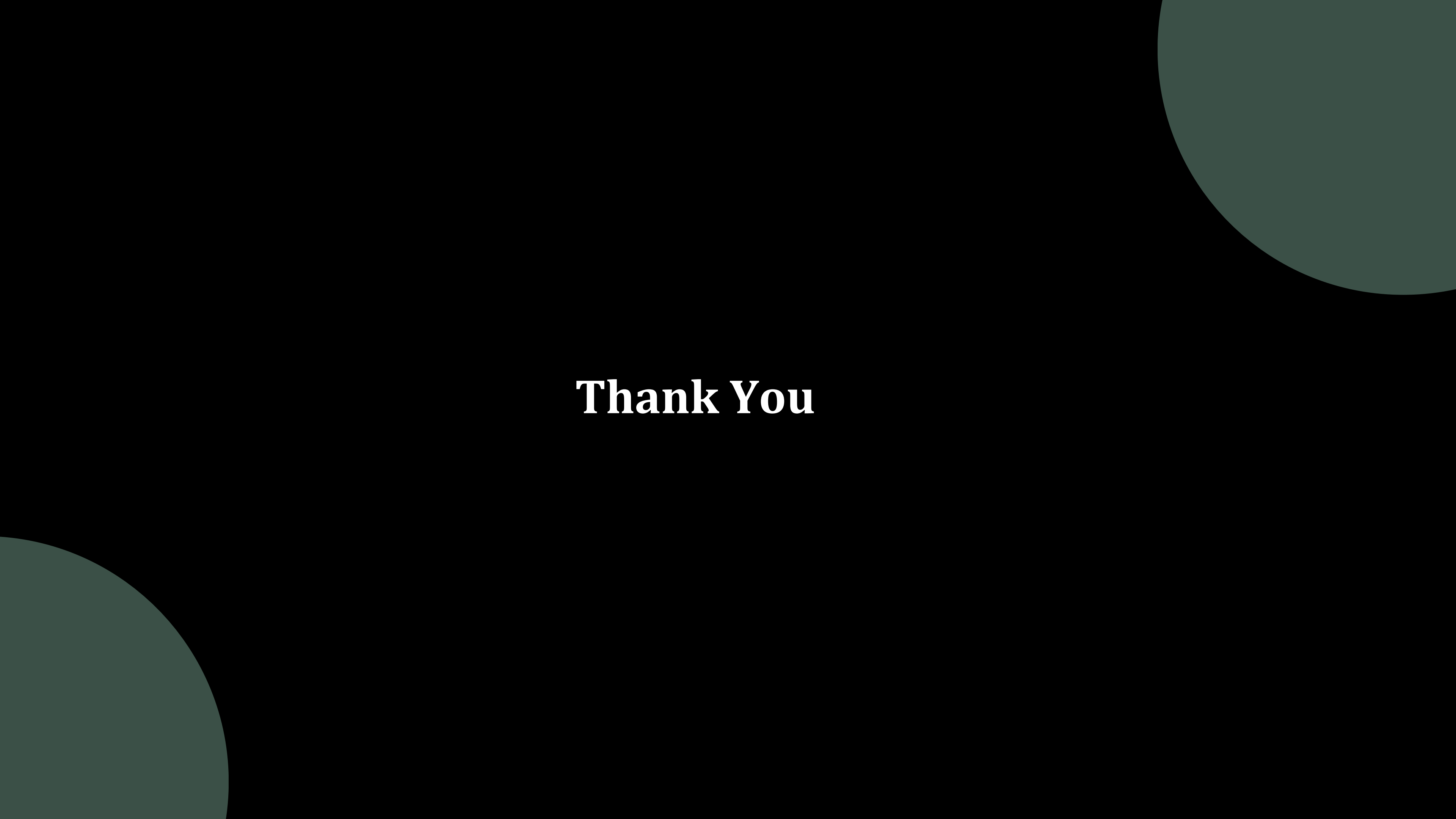
Every entry needs tenant approval or a valid pre-approval/absence password

4 Easier tracking

Append-only visit_log gives a full, searchable visitor history

5 Modular React UI

Reusable components (Modal, StatCard, Toast) keep the codebase maintainable



Thank You